

Avaliação no ScienceBenchmark (zero-shot)

O checkpoint treinado no Spider (t5-base + RGAT, 5 épocas, 8577 exemplos de treino, resultado no dev set do Spider: 41,9% *exact match* / 44,8% *execution accuracy*) foi avaliado, sem nenhum re-treinamento, sobre os 299 exemplos do split de desenvolvimento do ScienceBenchmark [?], distribuídos em três bancos de domínios distintos: **cordis** (projetos de pesquisa europeus), **oncomx** (biomedicina) e **sdss** (astronomia).

1.1 Resultado agregado

Table 1: Resultado do eval zero-shot do checkpoint Spider-treinado no ScienceBenchmark, por banco.

Banco	n	exact match	exec	Exemplos avaliados
cordis	100	12,0%	15,0%	100/100
oncomx	99	14,1%	11,1%	99/99 (exec: 97/99)
sdss	94	0,0%	0,0%	90/94 ^a
Total (agregado)	293	9,0%	8,9%	exact: 289/293, exec: 287/293

^a4 exemplos do sdss descartados da métrica: parser SQL do Spider (`third_party/spider/process_sql.py`) não suporta parênteses agrupando condições no `WHERE`, um padrão ausente no SQL gold do Spider mas presente no do ScienceBenchmark.

Para efeito de comparação, o mesmo checkpoint avaliado no dev set do Spider (1034 exemplos, mesmo domínio de treino) obteve 41,9% *exact match* e 44,8% *exec* – uma queda de ~33 pontos percentuais ao mudar de domínio sem re-treinamento, consistente com a expectativa de que arquiteturas graph-aware treinadas em um conjunto de bancos pequenos e homogêneos (Spider, ~5,3 tabelas/banco em média) generalizam de forma limitada para bancos maiores e de domínio mais específico (ScienceBenchmark, 19–25 tabelas no **cordis**, colunas de medição numérica pouco usuais no **sdss**).

1.2 Quebra por dificuldade

Usando a classificação de dificuldade do avaliador oficial do Spider (*easy/medium/hard/extra*, baseada em número de componentes SQL, condições e agregações), o *exact match* cai a zero em *hard* e *extra* nos três bancos, sem exceção:

Table 2: Exact match por nível de dificuldade e banco.

Banco	easy	medium	hard	extra
cordis (n=100)	32,0% (n=25)	11,4% (n=35)	0% (n=19)	0% (n=21)
oncomx (n=99)	42,9% (n=21)	13,9% (n=36)	0% (n=22)	0% (n=20)
sdss (n=90)	0% (n=6)	0% (n=28)	0% (n=20)	0% (n=36)

O padrão observado sugere duas causas de queda distintas e sobrepostas: (i) uma limitação de *capacidade*, comum aos três bancos – o modelo t5-base resolve consultas simples mas não lida com consultas complexas (múltiplos JOINS, subconsultas, agregações compostas) fora do domínio de treino; e (ii) uma limitação de *transferência de domínio*, específica do **sdss** – o resultado é zero mesmo nas consultas *easy*, o que não ocorre em **cordis/oncomx**, indicando que o vocabulário e a estrutura de esquema do **sdss** (medições fotométricas, nomenclatura astronômica) são suficientemente distantes do domínio de treino a ponto de comprometer até os casos mais simples.

Uma observação metodológica reforça essa segunda leitura: o tamanho do esquema (contagem de nós do grafo RGAT) e o banco de dados são quase colineares nesta amostra – **cordis/oncomx** sempre excedem 512 nós, **sdss** está sempre abaixo. Como **sdss** é o banco com o *menor* esquema em número de nós e ainda assim o único com resultado nulo em todos os níveis de dificuldade, o fator determinante da queda não parece ser o tamanho do esquema, e sim a distância de domínio/vocabulário em relação ao Spider.

1.3 Análise qualitativa dos erros

Das 263 previsões incorretas (excluindo os 4 exemplos de `sdss` sem gold parseável), 179 (68%) foram inicialmente classificadas como “SQL sintaticamente inválido” pelo parser oficial do Spider (`third_party/spider/process_sql.py`) – ou seja, o modelo nem produziu uma consulta que o parser conseguisse interpretar como AST SQL.

Table 3: Distribuição das categorias de erro (previsões incorretas, exact match = 0).

Categoria	Total	cordis	oncomx	sdss
SQL sintaticamente inválido	179	56	58	65
Tabela errada	50	13	12	25
Coluna errada	19	9	10	0
Outro erro semântico	10	5	5	0
Agregação errada	5	5	0	0

Como o parser oficial do Spider já se mostrou limitado a uma gramática SQL restrita neste trabalho (cf. o caso de parênteses agrupando condições no `WHERE`, seção anterior), investigou-se se parte dessas 179 previsões “inválidas” seriam, na verdade, SQLite sintaticamente válido que o parser simplesmente não reconhece. Cada uma das 179 previsões foi executada diretamente contra o banco SQLite real correspondente (`sqlite3`, sem passar pelo parser AST do Spider). O resultado foi conclusivo: **nenhuma das 179 (0%) executou com sucesso** – todas produziram um erro real do SQLite, predominantemente no `such table` ou no `such column` (por exemplo, `institutions.uncs_id` em vez de `unics_id`, ou um `join` com a tabela inexistente `subjects` em vez de `subject_areas`). Isso descarta a hipótese de artefato de parser para esta categoria: a maioria dos erros do modelo no ScienceBenchmark é, de fato, alucinação de nomes de tabela/coluna inexistentes no esquema ou SQL genuinamente malformado, não uma limitação da ferramenta de avaliação.

1.4 Decodificação restrita por gramática (PICARD)

Dado que a maioria dos erros é SQL sintaticamente inválido, testou-se se a decodificação restrita por gramática do PICARD [?] – que impede o modelo de gerar tokens que levariam a uma consulta inválida – reduziria essa taxa. O eval foi refeito com `use_picard=true`, `launch_picard=true`, mantendo os demais parâmetros idênticos.

Resultado: as 293 previsões geradas com PICARD ativo são *idênticas, token a token*, às geradas sem PICARD – exact match e exec saíram exatamente iguais aos valores da Tabela 1. Investigando a causa, constatou-se que o PICARD nunca chega a interceptar a geração nesta configuração: em `seq2seq/models/graphix/rgat_picard` um ajuste anterior (necessário para corrigir a resolução da classe do modelo, que sem ele carregava um T5 genérico em vez da variante com RGAT) passou a importar a classe do modelo diretamente, contornando o mecanismo (`model_cls_wrapper`) pelo qual o `generate()` restrito do PICARD normalmente seria aplicado. O servidor PICARD chega a subir e ficar ativo durante o eval, mas nunca é de fato consultado pela geração – confirmado tanto pela ausência total de qualquer menção a “picard”/“thrift” nos logs do eval quanto pela identidade exata das previsões. Trata-se de uma lacuna de integração pré-existente, não avaliada anteriormente porque o eval com PICARD nunca havia sido exercitado de ponta a ponta neste projeto; o conserto exigiria combinar os dois mecanismos de substituição de classe (RGAT-aware e PICARD-aware) em vez de tratá-los como alternativas mutuamente exclusivas, o que fica registrado aqui como limitação conhecida e trabalho futuro.

1.5 Busca em feixe (*beam search*, `num_beams=4`)

Como variação de decodificação adicional (mantendo o checkpoint zero-shot inalterado), o eval foi refeito com `num_beams=4` no lugar da decodificação gulosa (`num_beams=1`) usada nos resultados acima.

Resultado: 9,34% exact match / 9,56% exec (289 exemplos pontuados para exact match, 287 para exec), `eval_loss=0,6885` – uma melhora pequena mas real sobre a decodificação gulosa (9,0%/8,87%), consistente com o ganho usual, porém modesto, de busca em feixe sobre geração gulosa em tarefas de geração estruturada.

2 Fine-tuning no ScienceBenchmark

A partir do mesmo checkpoint zero-shot, foram conduzidos dois experimentos de ajuste fino sobre os dados de treino do ScienceBenchmark (4732 exemplos, seed+synth combinados): um fine-tuning completo (todas as épocas sobre o maior subconjunto de dados que a GPU disponível comporta) e um fine-tuning *few-shot* (poucas centenas de exemplos, mais épocas). Ambos foram avaliados no mesmo split de dev usado no zero-shot, seguindo a mesma metodologia de scoring (exact match + execução, mesmos filtros de segurança contra travamentos de execução no *sdss*).

Restrição de hardware determinante: os experimentos rodaram numa única GPU de consumidor (NVIDIA GeForce GTX 1070, 8 GB de VRAM, sem tensor cores), o mesmo hardware usado para os evals acima. Diferente do Spider (esquemas pequenos, $\sim 5,3$ tabelas/banco em média), os grafos RGAT do ScienceBenchmark chegam a mais de 1300 nós por exemplo, o que por si só é suficiente para saturar 8 GB de VRAM durante o treino (fase que aloca ativações para retropropagação, ao contrário do eval, que só faz inferência).

2.1 Fine-tuning completo

Configuração: 3 épocas, *batch_size*=1 com *gradient_accumulation_steps*=32 (lote efetivo 32), taxa de aprendizado 5×10^{-5} com *adafactor*, 330 passos de otimização totais. Por restrição de VRAM, exemplos cujo grafo RGAT excede 1000 nós foram descartados do treino – isso excluiu **1165 dos 1165 exemplos de oncomx** presentes no split de treino após o primeiro filtro (praticamente o banco inteiro; ver limitação (iii) abaixo), mantendo *cordis* e *sdss* quase integralmente (3540 dos 4732 exemplos totais permaneceram).

Table 4: Fine-tuning completo: resultado final no dev set completo (289–293 exemplos, sem o filtro de tamanho de grafo aplicado ao treino).

Configuração	exact match	exec	eval_loss
Zero-shot (referência)	9,0%	8,87%	–
Fine-tuning completo	0,0%	0,34%	4,2321

O resultado é **pior que o zero-shot**, apesar do *loss* de treino ter caído de forma saudável e monotônica até 0,469 ao final das 3 épocas. A causa raiz identificada: o modelo nunca viu nenhum exemplo de *oncomx* durante o treino (excluído por VRAM), mas o dev set de avaliação final inclui *oncomx* normalmente – o desempenho ruim nesse domínio nunca visto arrasta a média agregada para baixo, revertendo o ganho que o fine-tuning trouxe em *cordis/sdss*. Esse é um achado metodológico central deste trabalho: **fine-tuning completo sob restrição severa de VRAM que exclui um dos domínios-alvo pode piorar a generalização agregada**, mesmo quando todos os indicadores de treino (perda) sugerem que o ajuste está funcionando.

2.2 Fine-tuning *few-shot*

Para tentar obter alguma representação de *oncomx* sem incorrer no mesmo custo proibitivo de VRAM ao longo de um treino completo, foi montado um segundo experimento com uma amostra pequena e deliberadamente balanceada entre os três bancos: 140 exemplos de *cordis* + 140 de *sdss* (amostragem aleatória) + 20 de *oncomx* (os 20 exemplos de menor grafo disponíveis, 1022–1025 nós – ver limitação (iii)), 300 exemplos no total, por 10 épocas (90 passos de otimização).

Table 5: Fine-tuning few-shot: resultado final no dev set completo.

Configuração	exact match	exec	eval_loss
Zero-shot (referência)	9,0%	8,87%	–
Fine-tuning completo	0,0%	0,34%	4,2321
Fine-tuning few-shot	0,0%	0,0%	4,8754

O resultado também ficou abaixo do zero-shot, e abaixo até do fine-tuning completo no exec. Diferente do fine-tuning completo, aqui não há exclusão sistemática de um banco (os três estão representados, ainda que `oncomx` de forma mínima) – a causa mais provável é volume de dados insuficiente (300 exemplos) combinado com uma limitação metodológica própria deste experimento, descrita em (iv) abaixo: o checkpoint final salvo corresponde a apenas 52% do treino planejado, não ao ponto de convergência real.

2.3 Correção do *gradient checkpointing* e fine-tuning completo revisado

A limitação (iii) acima – exclusão total de `oncomx` do treino por falta de VRAM – motivou uma segunda iteração deste trabalho, buscando incluir os três domínios sem reduzir `max_source_length`. O mecanismo natural para isso é o *gradient checkpointing* do PyTorch (`torch.utils.checkpoint`): em vez de manter em memória todas as ativações intermediárias da rede para a retropropagação, ele as recalcula sob demanda durante o *backward pass*, trocando tempo de computação por memória. A flag já existia em `configs/train.json`, mas estava sem efeito para o modelo com RGAT – dois problemas distintos, ambos corrigidos:

- A classe `Model` em `seq2seq/models/graphix/rgat.py` é um invólucro (*wrapper*) que guarda o T5 com RGAT de fato como um atributo interno (`self.pretrain_model`), em vez de herdar sua hierarquia de camadas. O método padrão `gradient_checkpointing_enable()` do HuggingFace percorre recursivamente as submódulos de uma classe para ativar o checkpointing em cada um – como o RGAT vive dentro de um atributo e não na hierarquia percorrida, a ativação nunca chegava lá. Corrigido delegando explicitamente a chamada ao `pretrain_model` interno.
- Mesmo com a delegação corrigida, o `T5Stack.forward()` (em `seq2seq/models/modeling_t5.py`) constrói, quando o checkpointing está ativo, uma função de encapsulamento (*closure*) para cada bloco que é passada a `torch.utils.checkpoint.checkpoint()` – e essa *closure* não repassava `graph_batch/relation_emb` ao bloco, fazendo o RGAT ser silenciosamente ignorado (ou falhar) sempre que o checkpointing estava ativo. Corrigido capturando esses dois objetos por *closure* do Python (não são tensores diferenciáveis que o mecanismo de checkpoint precise gerenciar, então isso é seguro tanto na chamada inicial quanto no recálculo do *backward*).

A correção foi validada com um teste de fumaça isolado sobre os seis maiores grafos de `oncomx` (1217–1308 nós, os que mais frequentemente causavam OOM antes), confirmando treino sem erros e perda decrescente normalmente.

Com o *gradient checkpointing* funcionando, o fine-tuning completo foi refeito com os mesmos hiperparâmetros da Seção 2.1 (3 épocas, `batch_size=1`, `gradient_accumulation_steps=32`, 5×10^{-5} com `adafactor`), desta vez sem nenhuma exclusão de exemplo por tamanho de grafo – os 4732 exemplos de treino (incluindo `oncomx` por completo) participaram do treino.

Table 6: Fine-tuning completo, antes e depois da correção do *gradient checkpointing*.

Configuração	exact match	exec	eval_loss	train_loss / tempo
Fine-tuning completo (sem <code>oncomx</code>)	0,0%	0,34%	4,2321	0,469 / –
Fine-tuning completo (com <code>oncomx</code> , gc)	0,0%	0,34%	1,6765	2,215 / 9h42min

O resultado é um achado genuinamente nuançado: incluir `oncomx` reduziu o `eval_loss` em quase $2,5 \times$ ($4,2321 \rightarrow 1,6765$) – uma melhora clara e esperada, já que o modelo agora vê exemplos do domínio que antes lhe era totalmente estranho. Mas essa melhora **não se traduziu em nenhum ganho nas métricas discretas**: `exact_match` permanece em 0% e `exec` praticamente inalterado (0,34%). Isso sugere que a hipótese da Seção 2.1 (exclusão de `oncomx` como causa única do resultado ruim) era *necessária mas não suficiente*: mesmo com todos os três domínios representados no treino, os erros do modelo continuam sendo do tipo que quebra `exact_match/exec` de forma binária – nome de tabela/coluna incorreto, estrutura de JOIN equivocada – e uma perda agregada mais baixa (que mede o quão "próxima" a distribuição de tokens gerada está do alvo, token a token) não impede esse tipo de erro categórico. As causas mais prováveis são

um orçamento de treino ainda pequeno (3 épocas sobre 4732 exemplos heterogêneos, contra 5 épocas sobre 8577 exemplos homogêneos no Spider original) e/ou uma limitação de capacidade do próprio t5-base diante da diversidade de esquemas do ScienceBenchmark, discutida em conjunto com o achado da próxima seção.

2.4 Baseline t5-small sem RGAT

Para isolar o efeito específico do RGAT dos demais fatores (tamanho do modelo, orçamento de treino, dados), foi conduzido um experimento de ablação: um T5 padrão, **sem nenhuma camada RGAT ou estrutura de grafo**, na variante `t5-small` (60M parâmetros, contra os ~ 220 M do `t5-base` usado em todos os experimentos anteriores). O restante do pipeline foi mantido idêntico – mesmo pré-processamento (a mesma serialização de esquema textual produzida pelas etapas de grafo, ainda que o grafo em si não seja consumido), mesmo split de treino/dev (4732/299 exemplos), mesmos hiperparâmetros (3 épocas, `batch_size=1`, `gradient_accumulation_steps=32`, 5×10^{-5} com `adafactor`). Uma nova classe `Model` (`seq2seq/models/graphix/plain.py`) carrega um `T5ForConditionalGeneration` padrão do HuggingFace e ignora completamente o `graph_batch`, selecionável por uma variável de ambiente (`GRAPHIX_MODEL_VARIANT=plain`) sem duplicar o restante do script de treino.

Table 7: t5-small sem RGAT vs. t5-base com RGAT (fine-tuning completo).

Configuração	exact match	exec	eval_loss	tempo de treino
t5-base + RGAT (com <code>oncomx</code> , gc)	0,0%	0,34%	1,6765	9h42min
t5-small sem RGAT (3 épocas)	1,38%	1,37%	1,1825	2h53min

O resultado é surpreendente: o modelo *menor* e *sem* qualquer estrutura de grafo superou o modelo com RGAT em todas as métricas – `exact match` $4\times$ maior (1,38% vs. 0,34% de `exec` no RGAT, comparação direta de `exec`: 1,37% vs. 0,34%, também $\sim 4\times$), `eval_loss` menor, e treino $3,4\times$ mais rápido (sem o custo de recomputação do *gradient checkpointing* nem a sobrecarga de propagar mensagens em grafos de até 1300 nós por exemplo). É importante notar que essa comparação **não é perfeitamente controlada**: `t5-small` e `t5-base+RGAT` diferem tanto na presença do RGAT quanto no tamanho do modelo base, então o experimento isola "RGAT vs. sem RGAT" apenas de forma aproximada, não com uma variável por vez. Ainda assim, o resultado é informativo na direção oposta à esperada – se o RGAT estivesse agregando valor líquido, seria razoável esperar que o modelo *maior e com estrutura adicional* superasse o menor e mais simples, não o contrário. Hipóteses para essa inversão:

- O RGAT deste projeto foi desenhado e ajustado para os esquemas pequenos e homogêneos do Spider ($\sim 5,3$ tabelas/banco); grafos de 1000+ nós do ScienceBenchmark estão 2–3 ordens de grandeza acima da escala para a qual a arquitetura foi validada, e a propagação de mensagens em grafos tão densos pode diluir sinal útil em vez de reforçá-lo.
- Um modelo maior e mais complexo (`t5-base+RGAT`) tipicamente precisa de mais dados e/ou mais épocas para superar um modelo menor em fine-tuning – com apenas 4732 exemplos e 3 épocas, o orçamento de treino pode ser insuficiente para que a capacidade extra do RGAT se traduza em vantagem prática, enquanto o `t5-small`, por ser mais simples, converge mais rapidamente para um padrão razoável com o mesmo orçamento.
- A complexidade adicional (mais parâmetros, mais uma fonte de gradiente via o *encoder* do grafo) pode estar introduzindo ruído de otimização que atrapalha a convergência dentro do orçamento de treino disponível, sem chegar a comprometer a estabilidade geral (o *loss* de treino do RGAT+t5-base ainda caiu de forma saudável, só não tão baixo quanto seria necessário para competir com o `t5-small` nas métricas discretas).

Este resultado não permite concluir que RGAT seja inerentemente inadequado para text-to-SQL sobre esquemas grandes – apenas que, **nas condições específicas deste trabalho** (hardware de 8 GB de VRAM, poucas horas de treino disponíveis, 4732 exemplos), a complexidade adicional do RGAT não se pagou em métricas discretas frente a um baseline T5 puro e menor, treinado com o mesmo orçamento de tempo.

2.5 Limitações metodológicas

Os dois experimentos de fine-tuning expuseram uma série de limitações, tanto de hardware quanto de configuração, documentadas aqui por transparência e como recomendação para trabalho futuro:

- (i) **Ausência de split de teste genuíno.** O ScienceBenchmark, como distribuído, só define `train` (`seed+synth`) e `dev`. O mesmo split de `dev` usado para validação durante o treino (seleção do "melhor" checkpoint) foi reutilizado como avaliação final – não há um terceiro split, cego durante todo o processo, para reportar como número de teste independente. O Spider, usado no treino-base, tem a mesma limitação estrutural (sem test set público), então isso é consistente com a metodologia herdada, mas permanece uma ressalva válida: o número final reportado pode estar levemente inflado em relação a um teste verdadeiramente cego.
- (ii) **`torch.cuda.empty_cache()` incondicional.** Uma chamada incondicional a `torch.cuda.empty_cache()` em `seq2seq/models/graphix/rgat_tuning.py::RGAT_Layer.forward`, uma vez por camada RGAT ($12\times$ por exemplo no t5-base), forçava uma sincronização CUDA completa a cada chamada. Esse código é compartilhado entre treino e avaliação (`modeling_t5.py` importa a mesma classe para ambos), então os resultados de zero-shot/PICARD/beam4 relatados acima provavelmente rodaram mais devagar do que precisavam – os números continuam válidos, só o tempo de execução foi maior que o necessário. O fix (só chamar `empty_cache()` sob OOM real, não preventivamente) foi validado por comparação direta: antes, nenhum passo de otimização completava em mais de uma hora; depois, o mesmo passo completava em minutos.
- (iii) **oncomx tem esquema uniformemente grande demais para 8 GB de VRAM.** Todos os 1165 exemplos de treino de oncomx produzem grafos RGAT entre 1022 e 1308 nós – não existe um subconjunto pequeno e seguro para amostrar. Mesmo os 20 exemplos "menores" usados no few-shot (1022–1025 nós) ainda estão acima do teto de 900–1000 nós que se mostrou confiável durante o treino completo, e provocaram OOMs individuais recorrentes durante o few-shot (capturados e pulados por um mecanismo de segurança, não por acaso – ver item v). Esta é uma restrição de hardware, não uma limitação de amostragem: nenhuma estratégia de seleção de exemplos resolve o problema sem reduzir `max_source_length` (o que exigiria reprocessar toda a base de grafos RGAT, fora do escopo deste trabalho). **Atualização:** esta restrição foi contornada numa iteração posterior deste trabalho habilitando corretamente o *gradient checkpointing* do PyTorch, o que permitiu incluir oncomx por completo no treino sem reduzir `max_source_length` – ver Seção 2.3.
- (iv) **Ausência de *warmup* de taxa de aprendizado no fine-tuning completo, e espaçamento de checkpoint excessivo no few-shot.** O fine-tuning completo usou `warmup_ratio=0`, fazendo a taxa de aprendizado saltar diretamente para 5×10^{-5} no primeiro passo – isso corresponde a um colapso temporário de geração (repetição de n-gramas, ex. "...loadversion = 301 and loadversion = 301 and..." até o limite de tokens), causando `eval_exact_match=0` em todos os checkpoints intermediários mesmo com o *loss* caindo normalmente. Um `no_repeat_ngram_size=3` foi adicionado durante o treino (reduzindo o tempo de geração em $5\times$ e eliminando os loops de n-grama), e o few-shot já nasceu com `warmup_ratio=0,1` e `no_repeat_ngram_size=3` desde o início – mas um padrão de repetição mais sutil (alternância entre tokens SentencePiece de tamanhos diferentes que decodificam para o mesmo caractere repetido, ex. "6666666...") persistiu, sugerindo que o modelo continuou sub-treinado para geração de literais numéricos mesmo ao final do processo. Separadamente, no few-shot, o intervalo entre checkpoints (`eval_steps=47`) foi ajustado no meio do treino para reduzir o tempo gasto em avaliação (cada rodada de eval levava até 111 minutos), mas isso fez com que só um checkpoint intermediário (passo 47 de 90, 52% do treino) fosse salvo antes do fim – o `load_best_model_at_end` carregou esse checkpoint como "melhor" por ser o único candidato com métrica registrada, e os pesos do passo 90 (o treino de fato mais avançado) nunca chegaram a ser persistidos em disco.
- (v) **Seleção de "melhor checkpoint" quebrada quando a métrica empata.** O fine-tuning completo usou `metric_for_best_model="exact_match"`. Como o colapso de geração descrito em (iv) manteve `exact_match=0,0` em *todos* os checkpoints intermediários, e a lógica de comparação do HuggingFace Trainer só substitui o "melhor" registrado quando a nova métrica é *estritamente* maior, o

primeiro checkpoint avaliado (passo 50 de 330, a versão *menos* treinada de todas) nunca foi superado e acabou sendo o modelo efetivamente salvo ao final do treino – os pesos do passo 300 (97% do treino) precisaram ser recuperados manualmente do diretório de checkpoint correspondente para a avaliação real reportada na Tabela 4. O few-shot corrigiu isso usando `metric_for_best_model="eval_loss"` com `greater_is_better=false` (uma métrica contínua, que decresceu de forma monotônica ao longo do treino), mas essa correção por si só não evitou a limitação (iv) relacionada ao espaçamento de checkpoints.

- (vi) **OOMs genuínos e travamento de driver de GPU.** Dois crashes de OOM (*out of memory*) genuínos interromperam o fine-tuning completo no meio do treino, e um deles coincidiu com um travamento completo do driver NVIDIA (`nvidia-smi` parou de reconhecer a placa), exigindo reinicialização completa da máquina – um reset de driver isolado não foi suficiente para restaurar o desempenho normal da GPU. Mitigações aplicadas: *checkpointing* a cada 50 passos (em vez de 500, o intervalo original herdado do treino no Spider, que nunca dispararia dentro de um treino de 330 passos) e uma captura de OOM por micro-batch em `seq2seq/utils/trainer.py::Seq2SeqTrainer.training_step` que pula apenas o exemplo problemático (zerando o gradiente parcialmente acumulado e liberando cache) em vez de derrubar o processo inteiro – validado diretamente durante o few-shot, que sobreviveu a dezenas de OOMs individuais de exemplos grandes de `oncomx` sem interrupção.

3 Comparação final

Table 8: Comparação de todas as configurações avaliadas no dev set do ScienceBenchmark.

Configuração	exact match	exec	Observação
Zero-shot (greedy, num_beams=1)	9,0%	8,87%	referência
Zero-shot + beam search (n=4)	9,34%	9,56%	melhor resultado geral
Zero-shot + PICARD	9,0%	8,87%	PICARD inerte (falha de integração, §1.4)
Fine-tuning completo, sem <code>oncomx</code> (3 épocas)	0,0%	0,34%	pior que zero-shot (§2.1)
Fine-tuning few-shot (300 ex., 10 épocas)	0,0%	0,0%	pior que zero-shot (§2.2)
Fine-tuning completo, com <code>oncomx</code> + gc (3 épocas)	0,0%	0,34%	eval_loss 2,5× menor, métricas discretas
Baseline t5-small sem RGAT (3 épocas)	1,38%	1,37%	supera todos os fine-tunings com RGAT (§2.3)

O achado central deste conjunto de experimentos permanece, em essência, **negativo em relação ao fine-tuning com RGAT**: em nenhuma das quatro tentativas de ajuste fino (completo sem `oncomx`, *few-shot*, completo com `oncomx` via *gradient checkpointing*) o modelo t5-base+RGAT superou o desempenho do checkpoint zero-shot com busca em feixe nas métricas discretas – mesmo depois de resolver a limitação de VRAM que impedia incluir todos os domínios de treino. A correção do *gradient checkpointing* (§2.3) trouxe uma melhora real e mensurável na perda de avaliação (2,5× menor), mas essa melhora não se propagou às métricas discretas – um resultado que, por si só, já é instrutivo: perda agregada e acerto exato/execução podem divergir significativamente quando os erros do modelo são predominantemente categóricos (nome de tabela/coluna errado) em vez de graduais.

O achado mais forte deste trabalho, no entanto, veio do baseline de ablação (§2.4): um **t5-small sem nenhuma estrutura de grafo**, treinado com o mesmo orçamento de dados e épocas, superou *todas* as configurações com RGAT fine-tunadas neste trabalho em exact match e exec, e o fez em menos de um terço do tempo de treino. Combinado com o resultado do zero-shot (onde o RGAT+t5-base é a única opção avaliada, pois não há um t5-small equivalente pré-treinado no Spider para comparação direta), a evidência deste trabalho aponta para uma conclusão qualificada: **nas condições de hardware e dados disponíveis, a complexidade adicional do RGAT não demonstrou vantagem sobre um T5 padrão no fine-tuning para o ScienceBenchmark** – um resultado negativo genuíno para a hipótese de que a estrutura de grafo ajudaria a generalizar para esquemas maiores e de domínio distinto do treino original. As limitações documentadas na Seção 2.5 (volume de dados, orçamento de épocas, ausência de um terceiro split de teste cego) continuam válidas e sugerem que essa conclusão é específica deste orçamento experimental, não

necessariamente uma propriedade fundamental da arquitetura RGAT – mas dentro do escopo efetivamente testado, o T5 puro foi tanto mais eficaz quanto mais barato de treinar.